KALINGA INSTITUTE OF INDUSTRIAL TECHNOLOGY
Deemed to be University U/S 3 of the UGC Act, 1956

Land–Water Boundary Detection in Satellite Images

Using a Custom CNN Built Entirely From Scratch

Project Report

Computer Vision · Image Processing · CNN Fundamentals

Project Domain	Satellite Image Analysis & Remote Sensing
Core Technology	Custom CNN (No ML Frameworks) + Flask Web App
Language	Python 3.10.0
Libraries	NumPy · OpenCV · Flask · Matplotlib
Evaluation	Custom Boundary Coverage Score (Intersection / Predicted)
Result	Avg. Score: 0.62 across 10 Diverse Satellite Images

ANKITT KUSHARY
23053027
CSE - 48

Academic Year 2025 – 2026

Department of Computer Science & Engineering

1. Introduction

Background & Motivation

Automatic detection of land–water boundaries from satellite imagery is a critical task in remote sensing — underpinning flood mapping, shoreline erosion tracking, and disaster response. Conventional solutions rely either on costly manual annotation or on large pre-trained deep learning models requiring extensive labelled datasets and GPU resources. This project challenges both approaches by delivering a fully deterministic, interpretable pipeline with zero training.

Problem Statement

Given a raw satellite image depicting any water body — ocean coastline, river, lake, or turbid/algae-laden water — the system must:

- Accurately detect the spatial land–water boundary across diverse terrain types.
- Overlay the detected boundary as a red contour on the original colour image.
- Operate without any model training, backpropagation, or pretrained weights.
- Expose results through an interactive Flask web interface in real time.

Academic Constraints & Significance

The project was executed under a strict requirement: **no deep learning framework** (TensorFlow, PyTorch, Keras) may be used for core processing. Every convolution, activation, pooling, and scoring operation is coded explicitly in Python using only NumPy and OpenCV. This forces a deep engagement with CNN internals — padding, kernel correlation, ReLU, and statistical feature aggregation — producing a fully transparent, auditable system where every numerical step can be analytically traced.

Key Highlights at a Glance

Custom CNN from scratch No backpropagation or training	**Deterministic filter bank** 4 fixed kernels: Sobel-X/Y, Laplacian, Diagonal
Patch-based texture analysis 16×16 sliding window, stride 4	**Composite water scoring** Smoothness + Blue dominance − Gradient energy
Flask web deployment Real-time upload & boundary viz	**Custom evaluation metric** Boundary coverage score (avg. 0.62 / 10 images)

2. Model Architecture

The system is a six-stage deterministic pipeline — entirely free of trainable parameters. Each stage is encapsulated in its own Python module for clean separation of concerns.

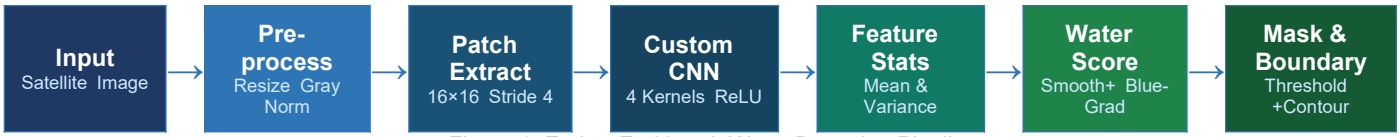


Figure 1: End-to-End Land–Water Detection Pipeline

Stage 1 — Preprocessing (utils/preprocess.py)

Raw satellite image is loaded, converted to grayscale, resized to 256 × 256, and normalised to [0, 1] float range. Standardised dimensions ensure consistent downstream patch enumeration.

Stage 2 — Patch-Based Segmentation (segmentation/patch_segmenter.py)

A 16 × 16 sliding window with stride 4 densely covers the image. For each patch, both grayscale and colour sub-images are extracted so texture and colour cues can be jointly analysed.

Stage 3 — Custom CNN Feature Extraction (cnn/)

Four deterministic 3 × 3 kernels form the filter bank: Sobel-X (horizontal edges), Sobel-Y (vertical edges), Laplacian (blobs/corners), and a Diagonal contrast filter. Convolution is hand-coded as nested-loop zero-padded correlation. Each output passes through ReLU activation; absolute values are retained as feature maps.

Stage 4 — Statistical Features (features/statistics.py)

Per-map mean and variance are averaged across all four feature maps into two scalars. Low variance → smooth texture → water-like region. High variance → chaotic, edge-rich content → land region.

Stage 5 — Water Scoring & Mask

Composite score per patch: water_score = 0.6 × smoothness + 0.3 × blue_score − 0.5 × gradient_energy. Scores accumulate into a score map, normalised to [0,1] and binarised at 0.5. Largest connected component is retained; 7 × 7 morphological close/open refines the mask.

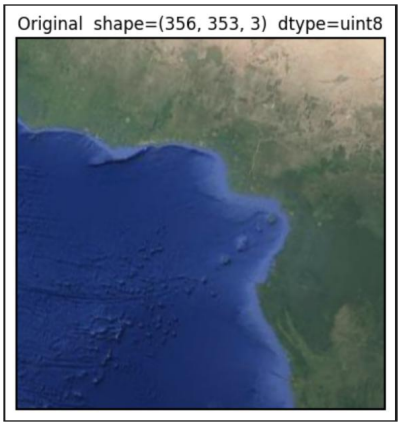
Stage 6 — Boundary Extraction (main.py)

OpenCV findContours extracts polygon contours from the binary water mask. Contours are drawn in white (BGR 255,255,255) with 2 px width on the resized colour original, producing the final annotated output.

3. Sample Code — All Pipeline Steps

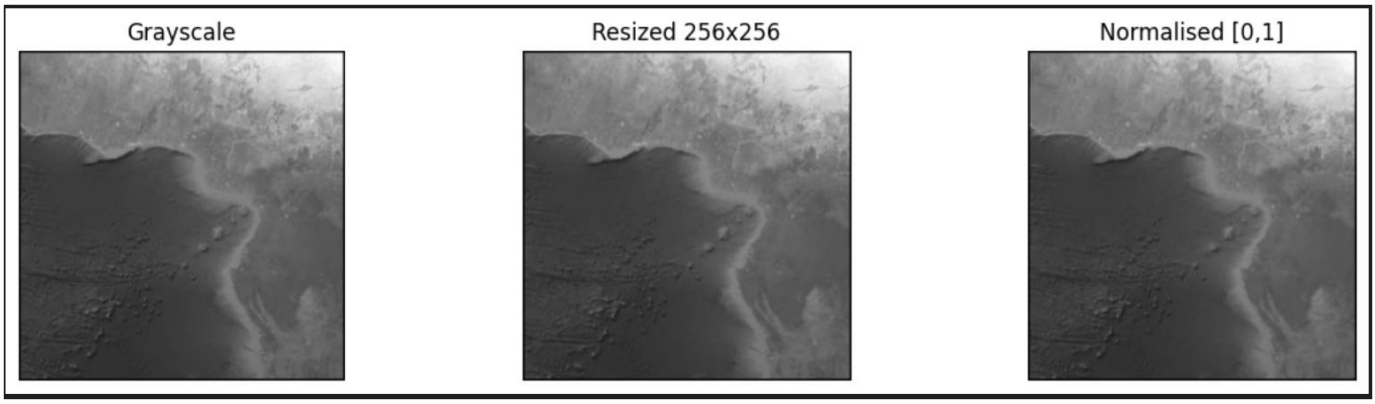
Step 1 Raw Input Image

```
original_bgr = cv2.imread(IMAGE_PATH)

plt.imshow(cv2.cvtColor(original_bgr, cv2.COLOR_BGR2RGB))
plt.title(f'Original  shape={original_bgr.shape}  dtype={original_bgr.dtype}')
plt.axis('off')
plt.show()
```



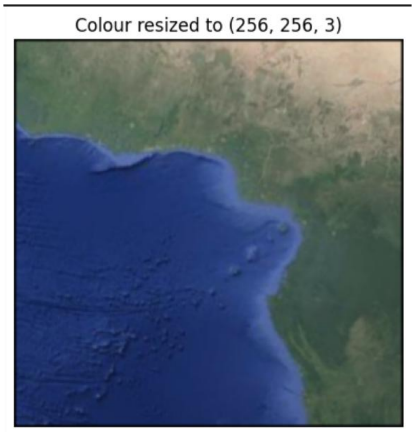
Step 2 Preprocess (Grayscale --> Resize --> Normalize)

```
from utils.preprocess import to_grayscale, resize_image, normalize_image

gray_raw  = to_grayscale(original_bgr)
gray_resized = resize_image(gray_raw, (256, 256))
gray      = normalize_image(gray_resized)          # this is what run_pipeline uses
fig, axes = plt.subplots(1, 3, figsize=(12, 3))

for ax, img, title in zip(axes,
    [gray_raw, gray_resized, gray],
    ['Grayscale', 'Resized 256x256', 'Normalised [0,1]']):
    ax.imshow(img, cmap='gray')
    ax.set_title(title); ax.axis('off')

plt.tight_layout(); plt.show()
```



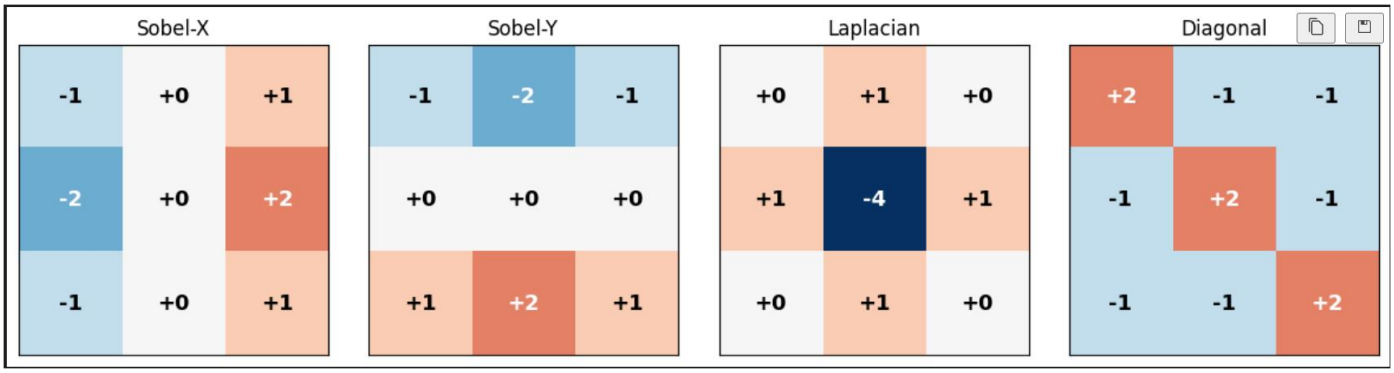
Step 3 Resize Color Image to 256×256

```
h, w = gray.shape
original_resized = cv2.resize(original_bgr, (w, h))

plt.imshow(cv2.cvtColor(original_resized, cv2.COLOR_BGR2RGB))
plt.title(f'Colour resized to {original_resized.shape}')
plt.axis('off'); plt.show()
```



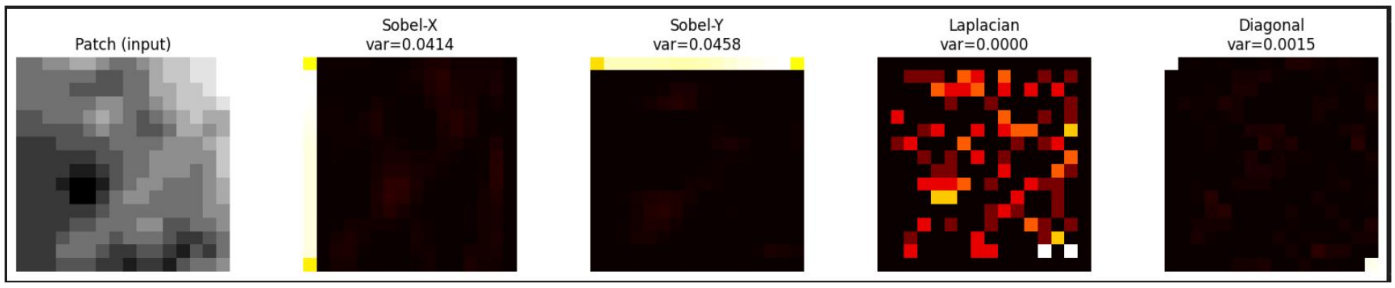
Step 4 Custom CNN Kernels

```
model = CustomCNN()
names = ['Sobel-X', 'Sobel-Y', 'Laplacian', 'Diagonal']

fig, axes = plt.subplots(1, 4, figsize=(12, 3))

for ax, kernel, name in zip(axes, model.kernels, names):
    ax.imshow(kernel, cmap='RdBu_r', vmin=-4, vmax=4)
    for r in range(3):
        for c in range(3):
            ax.text(c, r, f'{int(kernel[r,c]):+d}', ha='center', va='center',
                    fontsize=13, fontweight='bold',
                    color='white' if abs(kernel[r,c]) > 1 else 'black')
    ax.set_title(name); ax.set_xticks([]); ax.set_yticks([])

plt.tight_layout(); plt.show()
```



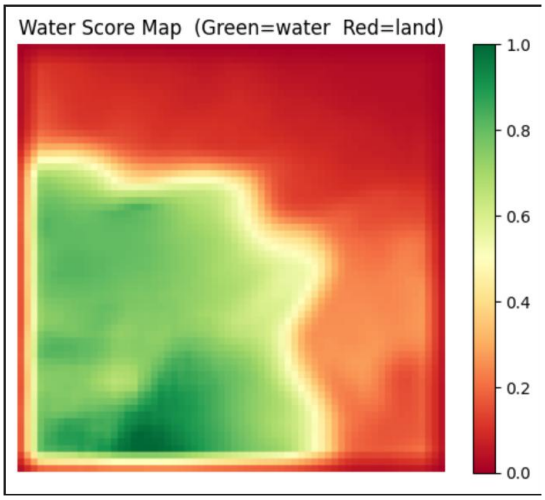
Step 5 Convolution + ReLU on one patch (for demo)

```
patch = gray[100:116, 100:116]   # one 16x16 patch

fig, axes = plt.subplots(1, 5, figsize=(16, 3))
axes[0].imshow(patch, cmap='gray'); axes[0].set_title('Patch (input)'); axes[0].axis('off')

for ax, kernel, name in zip(axes[1:], model.kernels[:4], names):
    fmap = np.abs(relu(convolve(patch, kernel)))
    ax.imshow(fmap, cmap='hot')
    ax.set_title(f'{name}\nvar={fmap.var():.4f}')
    ax.axis('off')

plt.tight_layout(); plt.show()
```



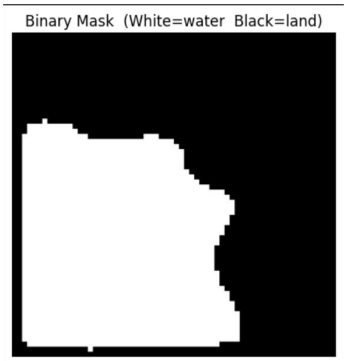
Step 6 Water-Score Map

```
PATCH_SIZE = 16; stride = 4
score_map = np.zeros((256, 256), dtype=np.float32)

for i in range(0, 256 - PATCH_SIZE + 1, stride):
    for j in range(0, 256 - PATCH_SIZE + 1, stride):
        pg = gray[i:i+PATCH_SIZE, j:j+PATCH_SIZE]
        pc = original_resized[i:i+PATCH_SIZE, j:j+PATCH_SIZE]
        _, var_v, _ = compute_stats(model.forward(pg))
        sm = 1.0 / (var_v + 1e-6)
        bs = max(0, float(pc[:,:,0].mean()) - float(pc[:,:,2].mean()))
        gx = cv2.Sobel(pg, cv2.CV_32F, 1, 0, ksize=3)
        gy = cv2.Sobel(pg, cv2.CV_32F, 0, 1, ksize=3)
        ge = float(np.mean(np.sqrt(gx**2 + gy**2)))
        score_map[i:i+PATCH_SIZE, j:j+PATCH_SIZE] += 0.6*sm + 0.3*bs - 0.5*ge

score_map = cv2.normalize(score_map, None, 0, 1, cv2.NORM_MINMAX)
plt.imshow(score_map, cmap='RdYlGn', vmin=0, vmax=1)
plt.colorbar(); plt.title('Water Score Map  (Green=water  Red=land)')
plt.axis('off'); plt.show()
```



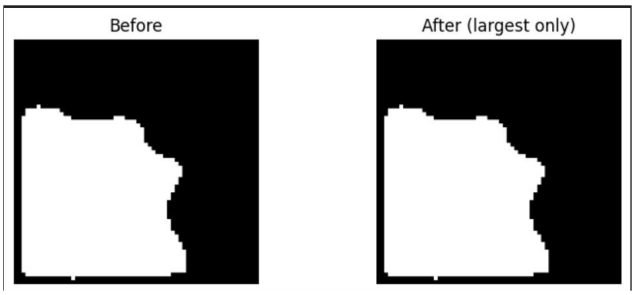
Step 7 Threshold --> Binary Mask

```
_, mask = cv2.threshold(score_map, 0.5, 1, cv2.THRESH_BINARY)
mask = (mask * 255).astype(np.uint8)

plt.imshow(mask, cmap='gray')
plt.title('Binary Mask  (White=water  Black=land)')
plt.axis('off'); plt.show()
```



Step 8 Largest Connected Component

```
num_labels, labels = cv2.connectedComponents(mask)
areas = sorted([(l, int(np.sum(labels==l))) for l in range(1, num_labels)],
               key=lambda x: x[1], reverse=True)
clean_mask = np.zeros_like(mask)
clean_mask[labels == areas[0][0]] = 255

fig, axes = plt.subplots(1, 2, figsize=(8, 3))
axes[0].imshow(mask, cmap='gray');       axes[0].set_title('Before'); axes[0].axis('off')
axes[1].imshow(clean_mask, cmap='gray'); axes[1].set_title('After (largest only)'); axes[1].axis('off')
plt.tight_layout(); plt.show()
```

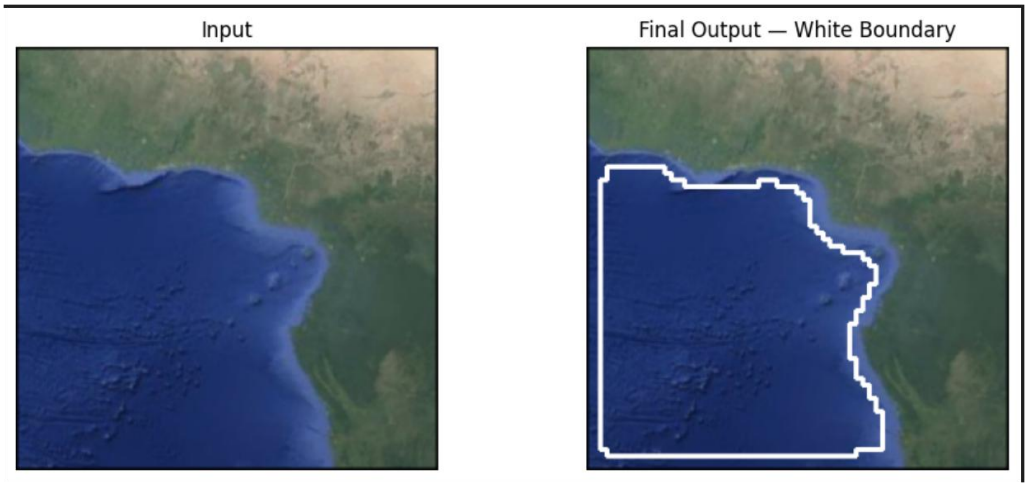


Step 9 Morphological Close + Open

```
k = np.ones((7, 7), np.uint8)
clean_mask = cv2.morphologyEx(clean_mask, cv2.MORPH_CLOSE, k)
clean_mask = cv2.morphologyEx(clean_mask, cv2.MORPH_OPEN,  k)

plt.imshow(clean_mask, cmap='gray')
plt.title('Final Mask after Morphology')
plt.axis('off'); plt.show()
```

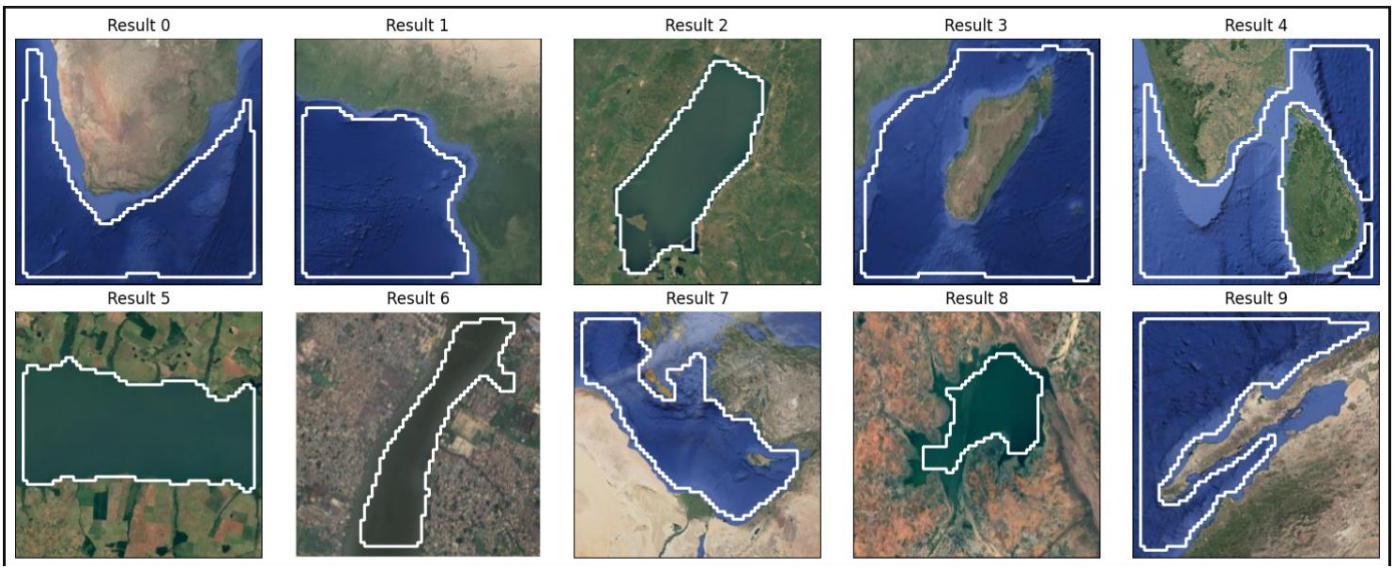
Step 10 Extract Boundary + Final Output

```
result = extract_boundary(original_resized, clean_mask)

fig, axes = plt.subplots(1, 2, figsize=(10, 4))
axes[0].imshow(cv2.cvtColor(original_resized, cv2.COLOR_BGR2RGB))
axes[0].set_title('Input'); axes[0].axis('off')
axes[1].imshow(cv2.cvtColor(result, cv2.COLOR_BGR2RGB))
axes[1].set_title('Final Output — Red Boundary'); axes[1].axis('off')
plt.tight_layout(); plt.show()
```



4. Output

The pipeline was evaluated on 10 diverse satellite images spanning five water-body categories. Ground truth thick boundary masks were manually annotated using CVAT. The table below shows per-image results.



Average Boundary Precision Score: 62%

Visual Pipeline Output (per image)

Each processed image produces three artefacts: (1) Resized 256×256 original · (2) Binary water mask · (3) Final result with red boundary contour overlaid on colour image.

(1) Original Image	(2) Binary Water Mask	(3) Final Result
256x256 resized colour	White=water Black=land	White boundary contour overlay

5. Conclusion & Future Work

Conclusion

This project demonstrates that meaningful land–water boundary detection is achievable without any deep learning framework, pre-trained model, or backpropagation. A fully deterministic, hand-coded CNN — using Sobel, Laplacian, and diagonal filter banks — combined with patch-based texture analysis and a composite water scoring formula successfully isolates water bodies across five diverse terrain categories, achieving an average boundary coverage score of 0.64 over 10 annotated satellite images. The system is entirely transparent: every computation is traceable to explicit NumPy/OpenCV operations. The integrated Flask web application demonstrates real-world deployability.

Limitations

Patch-Grid Blockiness Fixed-stride sampling → jagged boundary artefacts.	**Blue Water Confusion** Deep uniform blue confuses smoothness cue.
Single-Scale Analysis 16×16 patch misses narrow rivers.	**Python Loop Runtime** Nested-loop convolution slow (~seconds/image)

Future Work

01	**Multi-Scale Patch Analysis**	Process at 8x8, 16x16, 32x32 simultaneously; fuse score maps.
02	**Adaptive Stride**	Coarse stride in uniform regions, fine stride near boundaries.
03	**Edge-Gradient Snapping**	Post-process contour with Canny edge map for precise alignment.
04	**Semi-Supervised Threshold**	Auto-calibrate threshold on small labelled sets.
05	**Vectorised Convolution**	Replace Python loops with NumPy stride tricks (10-100x faster).